AI DevOps Incident Post-Mortem Report

Severity: MEDIUM

Service: Server

Post-Mortem Report: Server Incident

1. Executive Summary

On [Date], our Server service experienced a medium-severity incident due to increased CPU usage, likely caused by a recent deployment. The issue required manual intervention, and our team took corrective actions to resolve the problem. This report provides a detailed analysis of the incident, its impact, and recommendations to prevent similar incidents in the future.

2. Timeline

- [Date]: Recent deployment was made to the Server service.
- [Time]: Increased CPU usage was detected, triggering alerts and notifications.
- [Time]: Automated systems diagnosed the issue and escalated it to the DevOps team due to the requirement for manual intervention.
- [Time]: DevOps team investigated the root cause, identified the recent deployment as the likely culprit, and began implementing corrective actions.
- [Time]: Resolution efforts were completed, and the Server service was restored to normal operation.

3. Root Cause

The root cause of the incident was increased CPU usage, which was likely due to the recent deployment made to the Server service. The deployment introduced changes that resulted in higher resource utilization, specifically CPU, than anticipated. This increase in CPU usage led to performance degradation and triggered alerts.

4. Impact

The incident had a medium impact on our services. The increased CPU usage resulted in slower response times for users of the Server service, potentially affecting their productivity and experience. However, due to the timely escalation and intervention, the incident was resolved before it could cause more severe disruptions or data loss.

5. Resolution

To resolve the incident, the DevOps team followed the recommended actions:
- Investigated recent deployment changes to identify potential bottlenecks or resource-intensive code.
- Checked for queries or processes that could be optimized for better performance.

- Considered and implemented adjustments to scale server resources temporarily to alleviate the CPU usage spike.
- Applied optimizations to the code and queries to reduce resource intensity.
- Monitored the service closely after the adjustments to ensure the issue was fully resolved and did not recur.

6. Prevention Recommendations

To prevent similar incidents in the future, we recommend the following:
- **Thorough Testing**: Implement more comprehensive testing of deployments, especially focusing on resource utilization under various load conditions.
- **Resource Monitoring**: Enhance monitoring of server resources (CPU, memory, etc.) to detect anomalies earlier, allowing for quicker response times.
- **Deployment Strategy**: Consider adopting a more gradual deployment strategy (e.g., canary releases) to mitigate the impact of problematic changes.
- **Code Reviews**: Conduct regular, in-depth code reviews to identify and address potential performance issues before they reach production.
- **Scaling and Performance Optimization**: Regularly review and optimize server configurations and resource allocations to ensure they are appropriately scaled for current and anticipated workloads.

By implementing these measures, we can reduce the likelihood of similar incidents occurring and improve the overall resilience and performance of our Server service.